

Literaturverzeichnis

- [1] Comaniciu, D., Ramesh, V. ; Meer, P. „Kernel-based object tracking“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 25.5 (2003), S. 564–577. DOI: [10.1109/TPAMI.2003.1195991](https://doi.org/10.1109/TPAMI.2003.1195991) (siehe S. 5).
- [2] Fazli, M. S. ; Quinn, S. *Object Tracking in a 360° View: A Novel Perspective on Bridging the Gap to Biomedical Advancements*. 2024. arXiv: [2412.01119](https://arxiv.org/abs/2412.01119) [cs.CV]. URL: <https://arxiv.org/abs/2412.01119> (siehe S. 4, 5, 9, 11).
- [3] Fazli, M. S., Vella, S. A., Moreno, S. N. J. ; Quinn, S. *Computational Motility Tracking of Calcium Dynamics in Toxoplasma gondii*. 2017. arXiv: [1708.01871](https://arxiv.org/abs/1708.01871) [q-bio.QM]. URL: <https://arxiv.org/abs/1708.01871> (siehe S. 6, 7).
- [4] Heylen, J., De Wolf, M., Dawagne, B., Proesmans, M., Van Gool, L., Abbeloos, W., Abdelkawy, H. ; Reino, D. O. „MonoCInIS: Camera Independent Monocular 3D Object Detection using Instance Segmentation“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, S. 923–934 (siehe S. 23).
- [5] Isard, M. ; Blake, A. „CONDENSATION—Conditional Density Propagation for Visual Tracking“. In: *International Journal of Computer Vision* 29.1 (1998), S. 5–28. ISSN: 1573-1405. DOI: [10.1023/A:1008078328650](https://doi.org/10.1023/A:1008078328650). URL: <https://doi.org/10.1023/A:1008078328650> (siehe S. 12, 13).
- [6] Kammel, S., Ziegler, J., Pitzer, B., Werling, M., Gindele, T., Jagzent, D., Schöder, J., Thuy, M., Goebel, M., Hundelshausen, F. von, Pink, O., Frese, C. ; Stiller, C. „Team AnnieWAY’s Autonomous System for the DARPA Urban Challenge 2007“. In: *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Hrsg. von Buehler, M., Iagnemma, K. ; Singh, S. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, S. 359–391. ISBN: 978-3-642-03991-1. DOI: [10.1007/978-3-642-03991-1_9](https://doi.org/10.1007/978-3-642-03991-1_9). URL: https://doi.org/10.1007/978-3-642-03991-1_9 (siehe S. 23).
- [7] Khanam, R. ; Hussain, M. *YOLOv11: An Overview of the Key Architectural Enhancements*. 2024. arXiv: [2410.17725](https://arxiv.org/abs/2410.17725) [cs.CV]. URL: <https://arxiv.org/abs/2410.17725> (siehe S. 15, 16).
- [8] Lang, A. H., Vora, S., Caesar, H., Zhou, L., Yang, J. ; Beijbom, O. *PointPillars: Fast Encoders for Object Detection from Point Clouds*. 2019. arXiv: [1812.05784](https://arxiv.org/abs/1812.05784) [cs.LG]. URL: <https://arxiv.org/abs/1812.05784> (siehe S. 17).
- [9] Lefevre, S., Bouton, E., Brouard, T. ; Vincent, N. „A new way to use hidden Markov models for object tracking in video sequences“. In: *Proceedings 2003 International Conference on Image Processing (Cat. No.03CH37429)*. Bd. 3. 2003, S. III–117. DOI: [10.1109/ICIP.2003.1247195](https://doi.org/10.1109/ICIP.2003.1247195) (siehe S. 13, 14).

- [10] Liu, Y. ; Xi, Z. *Proceedings of 2021 Chinese Intelligent Automation Conference*. 2021. URL: <https://doi.org/10.1007/978-981-16-6372-7> (siehe S. 13).
- [11] Lowe, D. G. „Distinctive Image Features from Scale-Invariant Keypoints“. In: *International Journal of Computer Vision* 60.2 (Nov. 2004), S. 91–110. ISSN: 1573-1405. DOI: [10.1023/B:VISI.0000029664.99615.94](https://doi.org/10.1023/B:VISI.0000029664.99615.94). URL: <https://doi.org/10.1023/B:VISI.0000029664.99615.94> (siehe S. 10, 11).
- [12] Luiten, J. *How to evaluate tracking with the HOTA metrics*. Medium blog post. 24. Feb. 2021. URL: <https://jonathonluiten.medium.com/how-to-evaluate-tracking-with-the-hota-metrics-754036d183e1> (online - zuletzt aufgerufen am 13.09.2025) (siehe S. 25–27).
- [13] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L. ; Leibe, B. „HOTA: A Higher Order Metric for Evaluating Multi-object Tracking“. In: *International Journal of Computer Vision* 129.2 (Feb. 2021), S. 548–578. ISSN: 1573-1405. DOI: [10.1007/s11263-020-01375-2](https://doi.org/10.1007/s11263-020-01375-2). URL: <https://doi.org/10.1007/s11263-020-01375-2> (siehe S. 24, 25, 27, 28).
- [14] NVIDIA. *VPI — Vision Programming Interface: DCF Tracker*. Version 4.0. URL: https://docs.nvidia.com/vpi/algo_dcf_tracker.html#paper_dcf (online - zuletzt aufgerufen am 12.09.2025) (siehe S. 19, 20).
- [15] NVIDIA Corporation. *Gst-nvtracker — DeepStream documentation*. DeepStream SDK documentation. Juli 2025. URL: https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_plugin_gst-nvtracker.html (online - zuletzt aufgerufen am 08.09.2025) (siehe S. 18–20).
- [16] Paragios, N. ; Deriche, R. „Geodesic active contours and level sets for the detection and tracking of moving objects“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.3 (2000), S. 266–280. DOI: [10.1109/34.841758](https://doi.org/10.1109/34.841758) (siehe S. 8).
- [17] Redmon, J., Divvala, S., Girshick, R. ; Farhadi, A. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: [1506.02640](https://arxiv.org/abs/1506.02640) [cs.CV]. URL: <https://arxiv.org/abs/1506.02640> (siehe S. 15).
- [18] RoboSense Technology Co., Ltd. *RoboSense E1R LiDAR Datasheet*. Datasheet. Accessed 2025-09-28. RoboSense Technology Co., Ltd., Jan. 2025. URL: https://cdn.roboSense.cn/20250423194054_31672.pdf (siehe S. 51).
- [19] Sapkota, R., Calero, M. F., Qureshi, R., Badgujar, C., Nepal, U., Poullose, A., Zeno, P., Vaddevolu, U. B. P., Khan, S., Shoman, M., Yan, H. ; Karkee, M. *YOLO advances to its genesis: a decadal and comprehensive review of the You Only Look Once (YOLO) series*. 2025. DOI: <https://doi.org/10.1007/s10462-025-11253-3>. arXiv: [2406.19407](https://arxiv.org/abs/2406.19407) [cs.CV]. URL: <https://arxiv.org/abs/2406.19407> (siehe S. 14, 16).
- [20] Soleimanitaleb, Z., Keyvanrad, M. A. ; Jafari, A. „Object Tracking Methods:A Review“. In: *2019 9th International Conference on Computer and Knowledge Engineering (ICCKE)* (2019), S. 282–288. URL: <https://api.semanticscholar.org/CorpusID:210930009> (siehe S. 3).

- [21] Stanford University. *ImageNet-1k. Large-Scale Visual Recognition Dataset*. Stanford Vision Lab. URL: <https://www.image-net.org/index.php> (online - zuletzt aufgerufen am 23.09.2025) (siehe S. 32).
- [22] Szeliski, R. „Image Processing“. In: *Computer Vision: Algorithms and Applications*. Cham: Springer International Publishing, 2022, S. 85–151. ISBN: 978-3-030-34372-9. DOI: [10.1007/978-3-030-34372-9_3](https://doi.org/10.1007/978-3-030-34372-9_3). URL: https://doi.org/10.1007/978-3-030-34372-9_3 (siehe S. 6–12).
- [23] ultralytics. *COCO-Dataset*. ultralytics yolo docs. 17. März 2025. URL: <https://docs.ultralytics.com/de/datasets/detect/coco/#coco-pretrained-models> (online - zuletzt aufgerufen am 16.09.2025) (siehe S. 30).
- [24] Visual Geometry Group, University of Oxford. *Training. Konfiguration*. ultralytics. URL: <https://docs.ultralytics.com/de/usage/cfg/#train-settings> (online - zuletzt aufgerufen am 23.09.2025) (siehe S. 34).
- [25] Wang, X., Qi, S., Zhao, J., Zhou, H., Zhang, S., Wang, G., Tu, K., Guo, S., Zhao, J., Li, J. ; Yang, M. *MCTrack: A Unified 3D Multi-Object Tracking Framework for Autonomous Driving*. 2024. arXiv: [2409.16149](https://arxiv.org/abs/2409.16149) [cs.CV]. URL: <https://arxiv.org/abs/2409.16149> (siehe S. 20–22, 41).
- [26] Yan, G., Liu, Z., Wang, C., Shi, C., Wei, P., Cai, X., Ma, T., Liu, Z., Zhong, Z., Liu, Y., Zhao, M., Ma, Z. ; Li, Y. „OpenCalib: A Multi-sensor Calibration Toolbox for Autonomous Driving“. In: *arXiv preprint arXiv:2205.14087* (2022) (siehe S. 52, 53).
- [27] Zhu, L. *PointPillars: A Simple PointPillars PyTorch Implementation for 3D LiDAR (KITTI) Detection*. GitHub repository. Version v1.0.0. 2025. URL: <https://github.com/zhulf0804/PointPillars> (online - zuletzt aufgerufen am 28.09.2025) (siehe S. 39).

2 Grundlagen und Stand der Technik

2.1 Objekt-Tracking

Visuelle Objektverfolgung bezeichnet das Problem, ein oder mehrere Objekte über aufeinanderfolgende Bilder hinweg zu identifizieren und ihre Position und Bewegung zu schätzen. Das Problem ist für zahlreiche Anwendungsfelder von zentraler Bedeutung - von der Verkehrsüberwachung über Robotik und Mensch-Maschine-Interaktion bis hin zu medizinischen Anwendungen und dem Tracking autonomer Fahrzeuge [20]. Im folgenden Abschnitt gibt eine Liste Auskunft über mögliche Schwierigkeiten beim OT:

- Beleuchtungsqualität
- Kontrast
- Auflösung
- Skalierung
- Verdeckung
- Bewegungsunschärfe

Um mit den verschiedenen Schwierigkeiten umgehen zu können, müssen die später aufgeführten Verfahren folgende Anforderungen besitzen:

Robustheit Robustheit bedeutet, dass das System selbst in schwierigen Verhältnissen wie beispielsweise geringem Kontrast, hoher Bewegungsunschärfe oder hohem Verdeckungsgrad zuverlässig arbeitet [20].

Anpassungsfähigkeit Um auf plötzliche und komplexe Bewegungsänderungen des Zielobjekts eingehen zu können, muss das Verfahren die aktuell sichtbaren Eigenschaften des Zielobjekts erkennen und verfolgen [20].

Echtzeit Die Verarbeitung eines Live-Videostreams setzt einen echtzeitfähigen Algorithmus voraus [20].

Mit stetig wachsenden Anforderungen und dem Drang zur Verbesserung werden die Algorithmen fortlaufend verbessert. Im Folgenden werden als Erstes die konventionellen Verfahren für OT vorgestellt. Darauf folgt die Weiterentwicklung mittels Deep-Learning (DL), welche auf den konventionellen Verfahren aufbauen.

2.1.1 Konventionelle Verfahren

Die bestehenden Verfahren lassen sich in drei Kategorien einordnen: traditionelle Bildverarbeitungstechniken, merkmalsbasierte Ansätze und probabilistisch-statistische Methoden. Im weiteren Verlauf werden die Verfahren jeweils einzeln vorgestellt und in ihren methodischen Besonderheiten erläutert [2].

Traditionelle Bildverarbeitung

Hierzu zählen Techniken wie Vorlagenabgleich, Hintergrundsubtraktion und Konturerkennung. Die Merkmale sind meist speziell für die aktuelle Aufgabenstellung angefertigt und kommen lediglich in Bereichen zum Einsatz, bei welchen die Rechenleistung begrenzt ist. Im Folgenden sind drei verbesserte Trackingmethoden beschrieben [2].

Farbtracking Bei dieser Methode wird die Farbverteilung in der Zielregion verwendet, um Objekte in den folgenden Bildausschnitten zu verfolgen. Zuerst wird der Hintergrund mittels eines Schwellenwertes entfernt. Das anschließend generierte Farbhistogramm dient als Objektmerkmal und wird kontinuierlich aktualisiert, um die Verfolgung nicht zu verlieren. Im Gegensatz zum Red-Green-Blue (RGB)-Farbraum ermöglicht die Verwendung des HSV-Farbraums die Trennung von Farbton und -intensität. Dies erhöht die Robustheit gegenüber Beleuchtungsänderungen während des Trackingprozesses. Probleme gibt es, wenn Objekt- und Hintergrundfarbe sich nicht eindeutig unterscheiden lassen oder Objekte teilweise verdeckt werden [2].

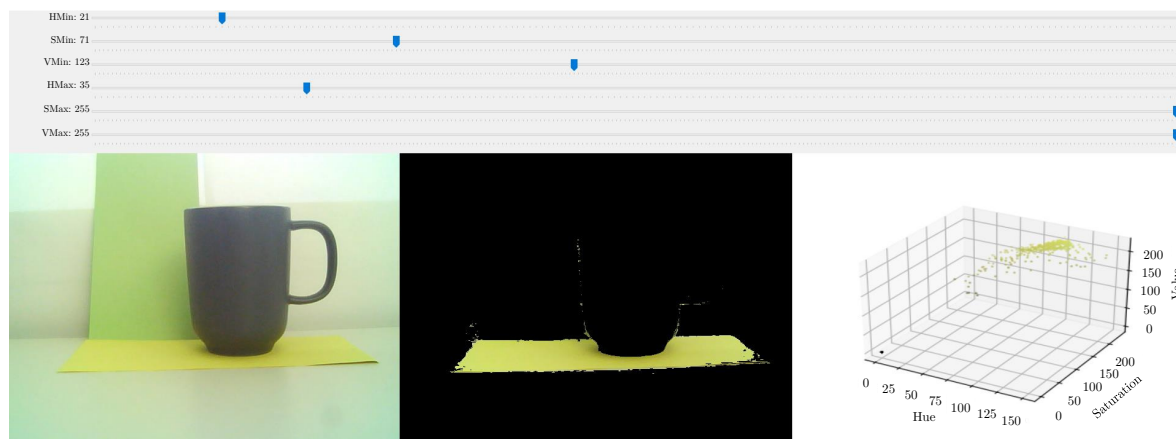


Bild 2.1. Anwendung HSV-Farbraumfilter

Kerneltracking Durch eine richtungsunabhängige Kernelfunktion lässt sich das Objekt räumlich darstellen, sodass mittels iterativer Suche die Ähnlichkeit zwischen Zielobjekt und potenziellen Positionen maximiert werden kann. Der Kernel entsteht aus dem ellipsenförmigen Zielobjekt, welches mittels der Breite und Höhe des Bildes auf

den Einheitskreis normalisiert wird. Die Randpixel des Kernels erhalten eine geringere Gewichtung, da sie häufiger Verdeckt und durch den Hintergrund gestört werden. Für das Zielmodell wird ein Merkmalshistogramm berechnet, welches die Wahrscheinlichkeit für ein Merkmal beschreibt. Ein Merkmal kann zum Beispiel die Farbe des Pixels sein, wodurch im Histogramm die Häufigkeit für das Auftreten des Farbwertes stehen würde. Die Wahrscheinlichkeitsdichtefunktion (Wdf) des Zielbereichs $\hat{q}_u$ steht in ►Gleichung 2.1 [2] [1].

$$\hat{q}_u = C \sum_{i=0}^n k \left(\|x_i^*\|^2 \right) \delta[b(x_i^*) - u] \quad (2.1)$$

Im Anschluss werden im nächsten Bild mögliche Zielkandidaten berechnet. Dabei ist die Wdf für Kandidaten $\hat{p}_u(y)$ wie in ►Gleichung 2.2 zu sehen fast identisch. Im Unterschied zu Gleichung 2.1 kann die Wdf für Kandidaten mit dem Bandbreitenfaktor skaliert werden. Dadurch erfolgt eine Anpassung des Suchbereichs [1].

$$\hat{p}_u(y) = C_h \sum_{i=0}^{n_h} k \left(\left\| \frac{y - x_i}{h} \right\|^2 \right) \delta[b(x_i) - u] \quad (2.2)$$

Nach der Berechnung der möglichen Kandidaten erfolgt der Vergleich durch den Bhattacharyya-Koeffizient $\hat{\rho}(y)$, welcher in ►Gleichung 2.3 abgebildet ist. Der Wertebereich des Koeffizienten liegt zwischen [0,1], wobei ein hoher Wert einer hohen Überlappung entspricht [1].

$$\hat{\rho}(y) \equiv \rho[\hat{p}(y), \hat{q}] = \sum_{u=1}^m \sqrt{\hat{p}_u(y) \hat{q}_u} \quad (2.3)$$

Anschließend wird im *Mean-Shift-Algorithmus* die Abstandsfunktion $d(y)$, welche in der ►Gleichung 2.4 abgebildet ist, benutzt. Dadurch wird aus dem Maximierungsproblem des Koeffizienten eine Minimierungsaufgabe der Funktion. Die Position wird so lange verändert, bis die Änderung unter einem Schwellwert ist. Dann wird die neue Zielposition übernommen und mit ihr mögliche Kandidaten im nächsten Bild gesucht [1].

$$d(y) = \sqrt{1 - \rho[\hat{p}(y), \hat{q}]} \quad (2.4)$$

Konturtracking Um die Kontur des Objekts zu erhalten, werden als erstes folgende Vorverarbeitungsschritte durchgeführt: [2]

- Histogrammausgleich
- Dilatation
- Erosion

Diese werden in den Folgenden Absätzen vorgestellt.

Histogrammausgleich | Ein Histogramm stellt die Summe eines jeden Pixelwertes im Bild in Form eines Balkendiagramms dar. Dieses wird mittels der kumulativen Verteilungsfunktion $c(I)$ wie in ►Gleichung 2.5 zu sehen ausgeglichen [22].

$$c(I) = \frac{1}{N} \sum_{i=0}^I h(i) \quad (2.5)$$

In ►Bild 2.2 ist ein vollständiger Histogrammausgleich dargestellt.

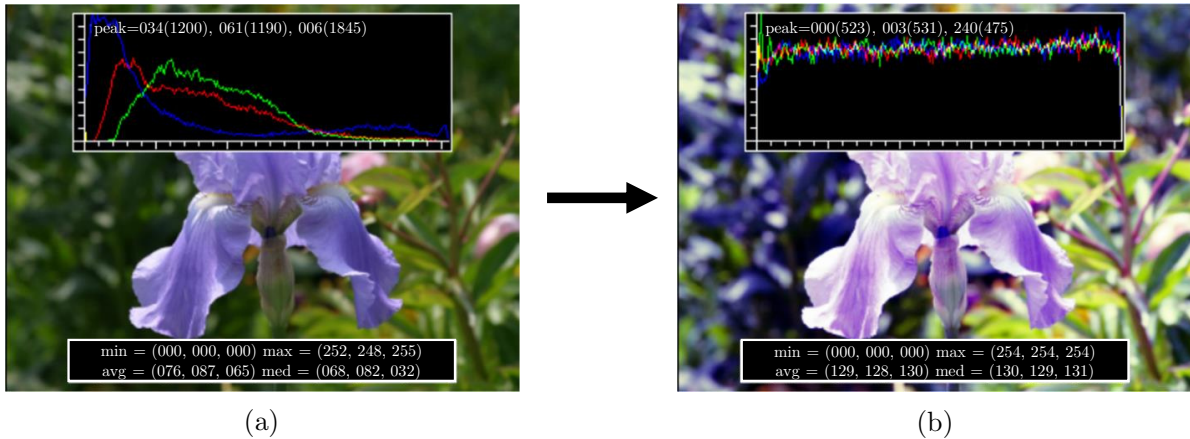


Bild 2.2. vollständiger Histogrammausgleich nach [22]: (a) original; (b) ausgeglichen

In der Praxis wird der Histogrammausgleich in einer Linearkombination mit der Identitätsabbildung verwendet. Die daraus entstehende Abbildungsfunktion ist in ►Gleichung 2.6 aufgeführt [22].

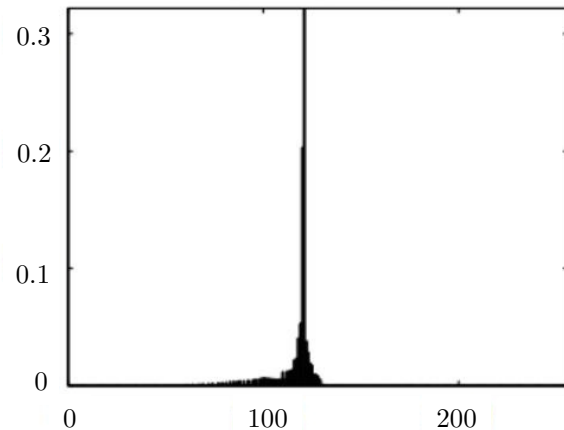
$$f(I) = \alpha c(I) + (1 - \alpha)I \quad (2.6)$$

Um die Konturen im gesamten Bild zu erhalten wird der Histogrammausgleich auf lokale Bildausschnitte angewendet. Somit werden mehrere Histogramme für ein Bild erstellt um die Beleuchtung und Kanten in verschiedenen Regionen anzupassen. Ein Beispiel für den lokalen Histogrammausgleich ist in ►Bild 2.3 dargestellt [3].

Originales Bild



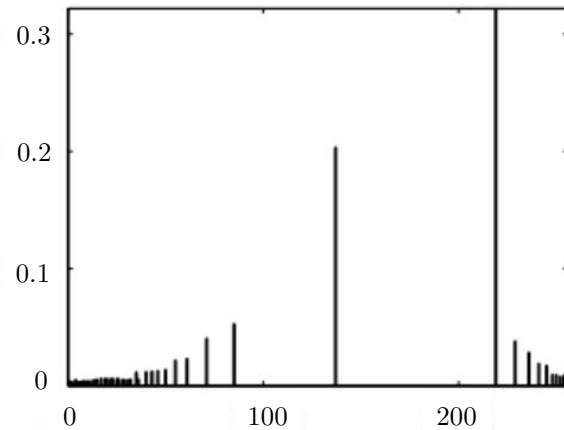
Originales Histogramm



Transformiertes Bild



Transformiertes Histogramm

**Bild 2.3.** lokaler Histogrammausgleich nach [3]

Dilatation & Erosion | Die Dilatation gehört wie die Erosion zu den morphologischen Operationen in der Bildverarbeitung. Dabei handelt es sich um Filter, die mithilfe einer Faltungsoperation auf das Bild angewendet werden. Ziel ist es, Kanten im Bild zu verstärken und gleichzeitig Hintergrundrauschen zu reduzieren. Bei der Dilatation werden Strukturen im Bild vergrößert. Dies führt dazu, dass kleine Lücken geschlossen und Kanten verbreitert werden. Die Erosion bewirkt das Gegenteil: Strukturen im Bild werden verkleinert. Dadurch verschwinden kleinere Objekte, und Kanten werden schmäler. Die Wirkung dieser beiden Operationen ist in ►Bild 2.4 veranschaulicht [22].

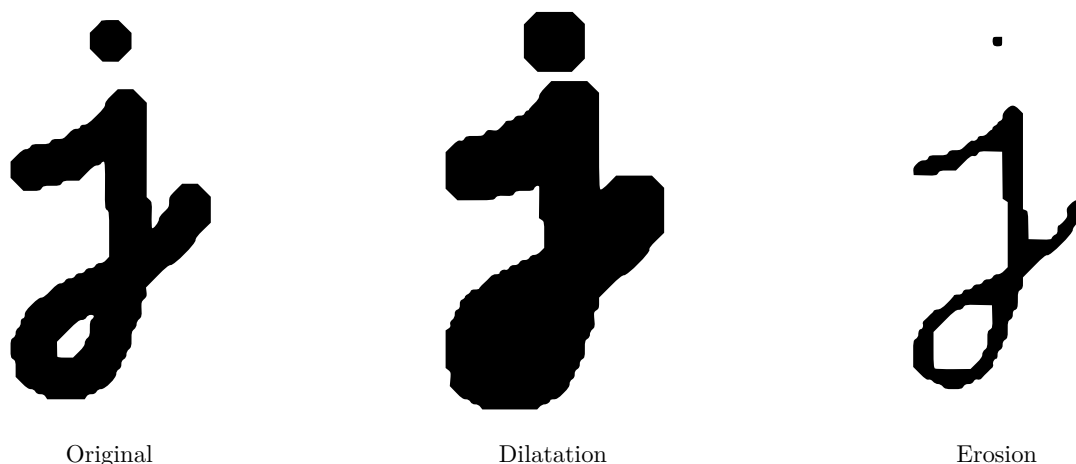


Bild 2.4. Erosion und Dilatation aus [22]

Nach der Vorverarbeitung werden die Konturen der Zielobjekte extrahiert. Diese Konturen werden anschließend in Bounding-Boxes (BB) umgewandelt, deren Mittelpunkte getrackt werden können. Zur Extraktion wird das Level-Sets-Verfahren im folgenden Absatz vorgestellt.

Level-Set-Theorie | In der Level-Set-Methode wird die Objektkontur nicht explizit als Kurve beschrieben, sondern implizit über die sogenannte Nullniveaumenge einer höherdimensionalen skalarwertigen Funktion. Diese Funktion ist auf dem gesamten Bild definiert und nimmt genau an den Punkten den Wert Null an, die auf der Kontur des Objekts liegen. Bereiche innerhalb des Objekts haben negative Werte, Bereiche außerhalb positive. Durch die zeitliche Weiterentwicklung dieser Funktion lässt sich die Kontur fortlaufend an die Objektgrenzen anpassen. Ein wesentlicher Vorteil dieser impliziten Darstellung besteht darin, dass topologische Änderungen der Kontur - wie das Zusammenwachsen oder Aufteilen von Objekten - automatisch und ohne zusätzliche Modellierungsschritte möglich sind. Durch die Minimierung einer energie-basierten Zielfunktion wird die Kontur über mehrere aufeinanderfolgende Bilder hinweg weiterentwickelt [22][16]. Dieser Prozess ist in ►Bild 2.5 dargestellt.

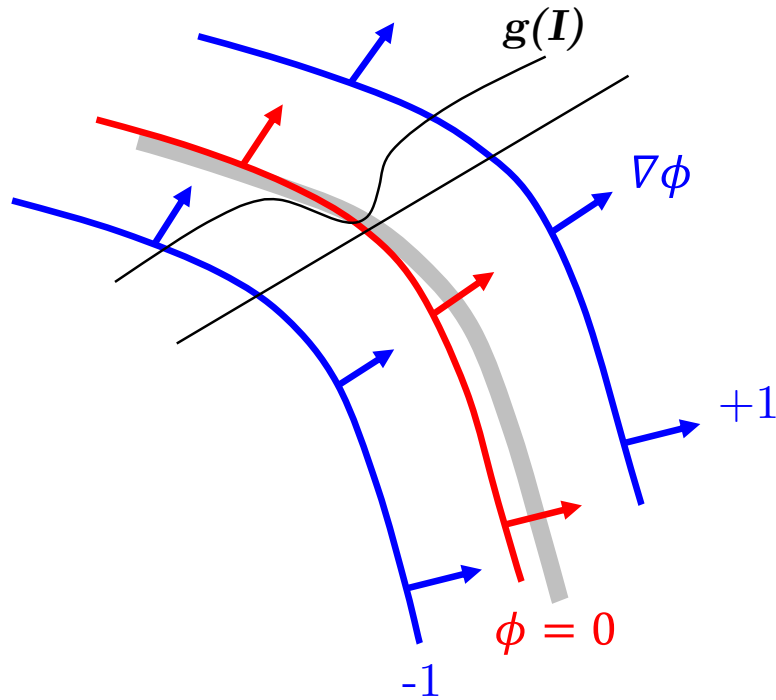


Bild 2.5. Level-Set evolution einer geodätisch aktiven Kontur nach [22]

Merkmals basierte-Tracking-Modelle

Diese Modelle basieren auf der Erkennung markanter Merkmale wie Ecken, Kanten oder struktureller Muster, die über mehrere Bilder hinweg verfolgt werden können. Dabei kommen Verfahren zum Einsatz, die den optischen Fluss eines Bildes analysieren oder Schlüsselpunkte von Objekten identifizieren. Diese Ansätze zeichnen sich durch ihre Robustheit gegenüber partieller Verdeckungen und wechselnden Beleuchtungsbedingungen aus. Im Folgenden werden die drei Merkmalsextraktionstechniken **optischer Fluss** und **SIFT & SURF** vorgestellt [2].

Optischer Fluss Die Bewegung von Intensitätsmustern in einer Bildfolge wird als optischer Fluss bezeichnet. Unter der Annahme, dass die Helligkeit eines Pixels bei Bewegung konstant bleibt, kann damit die Bewegung von Objekten zwischen aufeinanderfolgenden Bildern verfolgt werden. Die Helligkeitskonstanzannahme führt zur optischen Flussbedingung gemäß ►Gleichung 2.7 [22].

$$I_x * u + I_y * v + I_t = 0 \quad (2.7)$$

Für den optischen Fluss, siehe ►Gleichung 2.8, wird diese quadriert und anschließend integriert [22].

$$E_{HS} = \int (I_x u + I_y v + I_t)^2 dx dy \quad (2.8)$$

Durch die Hinzunahme der Glattheitsbedingung ist die vorher unterbestimmte Gleichung lösbar und aus der Minimierung der Energie folgt die Bewegung. Ein weiteres etabliertes Verfahren ist der Lucas-Kanade-Ansatz, der den Fluss lokal in kleinen Bildausschnitten schätzt und besonders bei kleinen Bewegungen gut funktioniert. Durch den Einsatz von Bildpyramiden lässt sich der Minimierungsprozess deutlich beschleunigen. Dabei wird zunächst eine niedrigere Bildauflösung verwendet, wodurch die Anzahl der zu verarbeitenden Pixel reduziert wird. Die geschätzte Bewegung auf dieser groben Ebene dient anschließend als Initialisierung für die nächste, höher aufgelöste Stufe [22].

SIFT & SURF Der Scale-Invariant-Feature-Transform (**SIFT**)-Algorithmus sucht Schlüsselpunkte in Bildern und beschreibt deren Umgebung mittels skalierungsunabhängiger Deskriptoren. Dafür wird das Bild mit verschiedenen Gaussfiltern unterschiedlicher Standardabweichungen gefaltet, wodurch die Skalenräume in ►Bild 2.6 entstehen. Eine Oktave besteht aus einer Reihe von Bildern, bei denen die Standardabweichung jeweils um einen konstanten Faktor erhöht wird. Nach Abschluss einer Oktave wird das Bild typischerweise um den Faktor zwei heruntergesamlet. Anschließend werden die Skalenräume subtrahiert um die *Difference-of-Gaussian (DOG)* zu erhalten. Die Extremwertsuche in den **DOG**-Ebenen ergibt die Merkmalskandidaten. Dazu wird ein Pixel mit seinen 26 direkten Nachbarn der 3x3-Matrix verglichen. Ein Kandidat wird nur ausgewählt, wenn er das Minimum oder das Maximum der Menge ist. Nach der Auswahl wird die Umgebung der Kandidaten überprüft. Punkte entlang einer Kante oder mit geringem Kontrast werden aussortiert. Im fünften Schritt erfolgt die Bestimmung der Orientierung des Merkmals. Dazu werden im Gegensatz zu anderen Methoden die lokalen Merkmale des Bildes verwendet. Der letzte Schritt berechnet den Merkmalsdeskriptor. Dazu wird die 16x16 Nachbarschaft des Merkmals in 16 4x4 Unterblöcke zerteilt. Jeder dieser Unterblöcke wird in ein 8-bin-Histogramm aufgeteilt, wodurch man einen Merkmalsvektor mit 128-Dimensionen erhält [11].

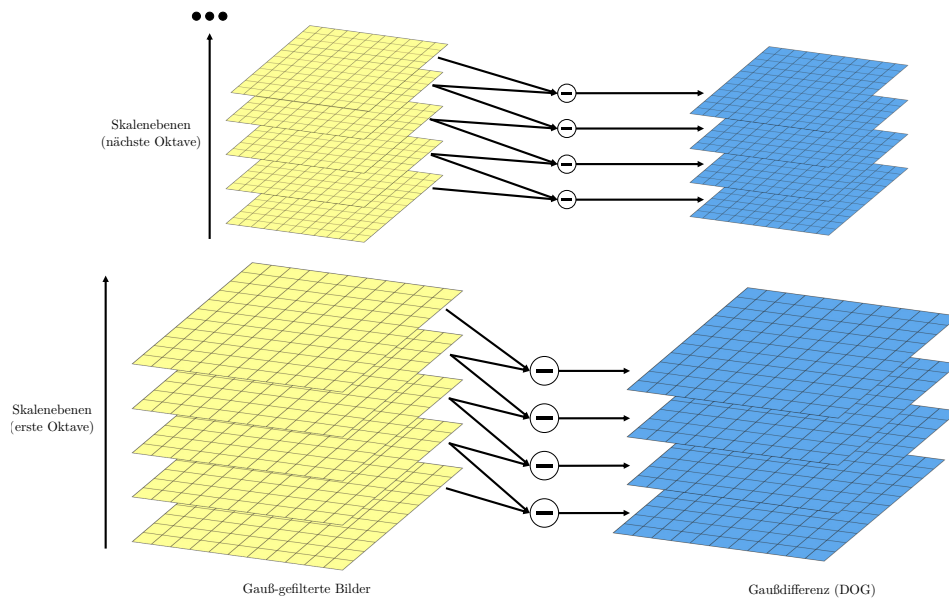


Bild 2.6. SIFT-Skalenräume nach [11]

Der Speeded-Up-Robust-Features (**SURF**)-Algorithmus optimiert das **SIFT**-Verfahren, indem er die rechenintensive **DOG**-Berechnung durch eine Näherung auf Basis der Determinante der Hesse-Matrix ersetzt und so die Komplexität deutlich reduziert [11].

Statistisch-probabilistische Methoden

Mittels Wahrscheinlichkeiten wird ein statistisches Modell aufgebaut, dass die zukünftige Position eines Objektes aus der vergangenen Trajektorie vorhersagt. Beispiele für diese Methoden sind der Kalman-Filter (**KF**), der Partikelfilter und die verborgenen Markov-Modelle. Diese Methoden zeichnen sich durch ihre besondere Eignung für Anwendungen aus, in denen Unsicherheiten und Störungen eine wesentliche Rolle spielen. Sie ermöglichen eine gezielte Ausblendung irrelevanter Informationen und erlauben gleichzeitig fundierte Vorhersagen, weshalb sie eine hohe Anpassungsfähigkeit und Robustheit besitzen [2].

Kalman-Filter Der **KF** besteht grundsätzlich aus den zwei Phasen Vorhersage und Aktualisierung. In der ersten Phase sagt der **KF** den Zustand X_{k+1} basierend auf dem aktuellen Zustand X_k und den bekannten Systemdynamiken voraus, siehe ►Gleichung 2.9. Diese werden in der Zustandsübergangsmatrix F_k und dem Prozessrauschen v_k beschrieben [22].

$$X_{k+1} = F_k X_k + v_k \quad (2.9)$$

In der zweiten Phase wird die Vorhersage durch die Hinzunahme neuer Messdaten verbessert. Die Aktualisierungsgleichung ist in ►Gleichung 2.10 dargestellt und beschreibt

mit Z_k die Messung zum aktuellen Zeitpunkt aus der Multiplikation von der Messmatrix H_k mit der vorhergesagten Messung und der Addition mit dem Messrauschen u_k [22].

$$Z_k = H_k X_k + u_k \quad (2.10)$$

Das Ziel des KF ist es, den Fehler zwischen der Vorhersage und Messung durch Anpassung der Vorhersage zu minimieren [22].

Partikelfilter Der Vorteil von Partikelfiltern liegt darin, dass sie die Wahrscheinlichkeitsverteilung eines Zustandsraums durch eine endliche Anzahl von Partikeln approximieren können. Jeder Partikel repräsentiert eine Hypothese über den Zustand des Systems und trägt ein Gewicht, das seine Relevanz angibt. Im Gegensatz zu klassischen KF, die auf einer unimodalen Verteilung basieren, können Partikelfilter multimodale Verteilungen darstellen und so komplexe Zustände besser erfassen. Für das Tracking in Szenarien mit starker Hintergrundstruktur und Verdeckungen wird der *Condensation*-Algorithmus eingesetzt. ►Bild 2.7 zeigt das Tracken einer Kopf-und-Schulter-Kontur. Dabei spiegeln die Maxima der Zustandsdichteverteilung die Position der Kontur wider. Der rechte Peak folgt über den zeitlichen Verlauf der Person zur linken Seite. Eine Kontur ist eine B-Spline-Kurve, welche die Schultern und Kopf umfasst. Aus den gewichteten Stichproben - auch Partikel bezeichnet - des vorherigen Zeitschritts wird per Resampling ein neuer Satz gezogen. Auf jeden Partikel wird ein gelerntes Dynamikmodell angewendet: ein deterministischer Drift und eine stochastische Diffusion. Für jede Konturhypothese werden entlang der Kurvennormalen Bildmerkmale geprüft und eine Beobachtungsdichte bestimmt. Diese dient zur Gewichtung der Partikel, welche in der nächsten Iteration erneut aussortiert werden. Dadurch konzentriert sich die Partikelmasse auf die wahrscheinlichsten Hypothesen und die Zustandsdichte entwickelt sich innerhalb weniger Schritte von einer annähernd normalverteilten Anfangsdichte zu einer multimodalen, siehe Bild 2.7 [22] [5].

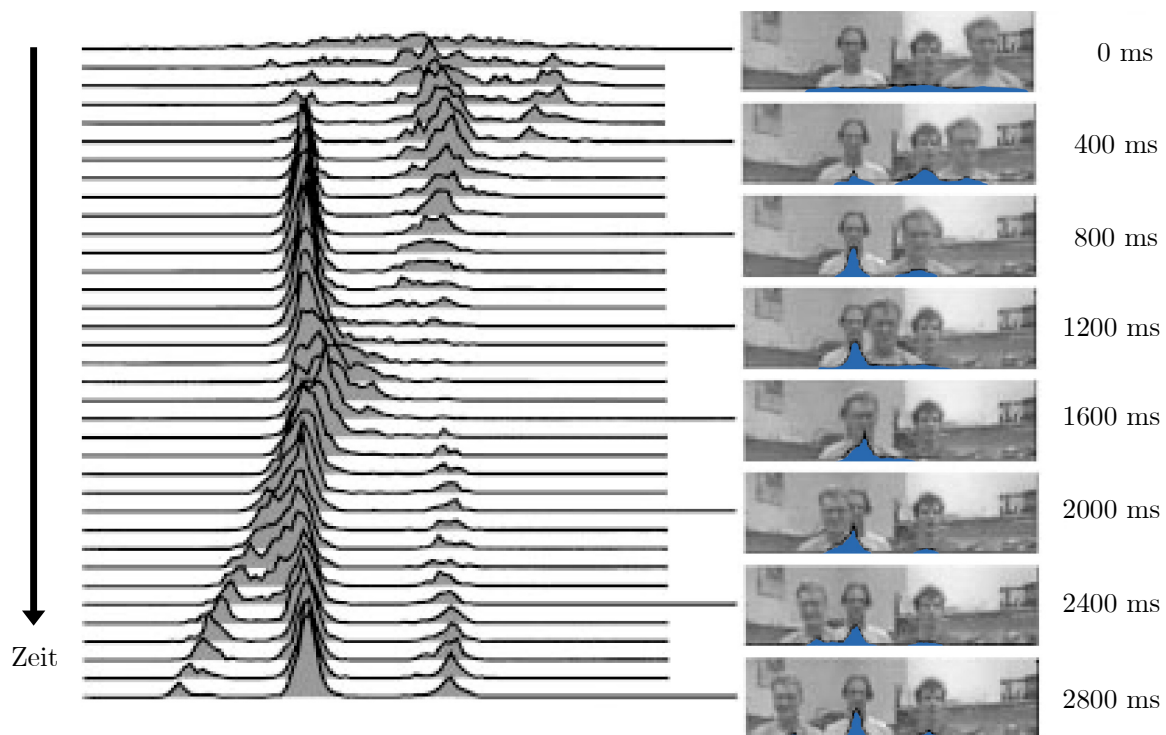


Bild 2.7. Approximierte Darstellung der Zustandsdichte projiziert auf die horizontale Translationsachse nach [5]

verborgene Markov-Modelle Das verborgene-Markov-Modell ([VMM](#)) modelliert die zeitliche Abfolge beobachtbarer Ereignisse, deren Entstehung von verborgenen inneren Zuständen abhängt. Es besteht also aus zwei Mengen von Zuständen - den verborgenen und den beobachtbaren - sowie aus den zugehörigen Parametern: der Zustandsübergangswahrscheinlichkeitsmatrix, die Beobachtungswahrscheinlichkeitsmatrix und dem Vektor der Anfangswahrscheinlichkeiten. Im dargestellten Verfahren, siehe ►Bild 2.8, erfolgt das Tracking in zwei Schritten. Zunächst wird die Objektposition durch einen simplen Bewegungsschätzer vorhergesagt (rot-weißer Rahmen). Anschließend wird im eingeschränkten Suchfenster mithilfe des gelernten [VMM](#) die exakte Position (schwarzes Fadenkreuz) bestimmt. Um festzustellen, ob das Objekt verloren gegangen ist, wird das Bild in Teilfenster zerlegt und diese mit dem Vorwärtsalgorithmus bewertet. Die dabei berechneten Scores geben an, wie gut die jeweiligen Fenster mit dem gelernten Objektmodell übereinstimmen [10] [9].



Bild 2.8. VMM-Tracking-Beispiel nach [9]

2.1.2 SOTA-Verfahren

In den folgenden Kapiteln werden aktuelle **SOTA**-Verfahren zur Objekterkennung und -verfolgung vorgestellt. Zur 2D Objekterkennung hat sich das **YOLO**-Framework etabliert, das durch seine Echtzeitfähigkeit und hohe Genauigkeit besticht. Für das Verarbeiten von Punktwolken hat sich **PointPillar** als effiziente Methode herauskristallisiert, welche die 3D-Objekterkennung in Echtzeit ermöglicht. Darauf folgt die Vorstellung des Software-Development-Kit (SDK) **DeepStream** von NVIDIA mit dem Tracking-Framework *nvtracker*, welches verschiedene Tracker-Backends für die 2D Multi-Objekt-Verfolgung bereitstellt. Zuletzt wird der 3D-Multi-Objekt-Tracker **MCTrack** vorgestellt, der erstmals **SOTA** Performance bei mehreren Benchmarks besitzt.

YOLO

You-Only-Look-Once (**YOLO**) ist ein einstufiges Objekterkennungsverfahren, das im Gegensatz zu zweistufigen Ansätzen den Regionsvorschlag und Klassifikation in einem einzigen Netzdurchlauf vereint. Dabei wird ein Bild in Raster unterteilt und danach für jede Rasterzelle **BB** und Klassenwahrscheinlichkeiten prognostiziert. Dieser Prozess ist in ►Bild 2.9 verdeutlicht [19].

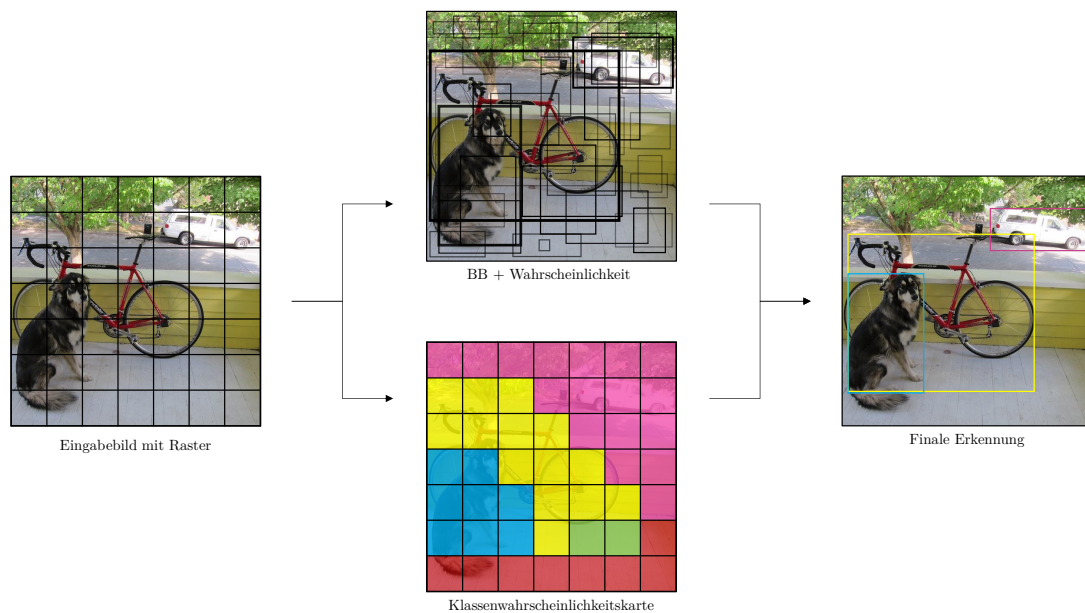


Bild 2.9. YOLO Beispielprozess nach [17]

Dieser Ansatz basiert auf einem Convolutional-Neural-Network (**CNN**), das durch aufeinanderfolgende Faltungen mit lernbaren Filtern sowie Zwischenschritte aus *Down-sampling* (Stride/Pooling) die räumliche Auflösung schrittweise verringert und dabei relevante Merkmale extrahiert und verdichtet. Historisch bestand *YOLOv1* nur aus dem eben beschriebenen Merkmalsextraktor (*Backbone*) und Fully-Connected-Layers (**FCLs**) zur Vorhersage der Wahrscheinlichkeiten und Koordinaten. Modernere Varianten wie *YOLOv11* strukturieren das **CNN**, siehe ►Bild 2.10, in *Backbone*, *Neck* und *Head* [17] [7].

Backbone Das *Backbone* besteht aus konventionellen Faltungsschichten und dem speziellen *C3k2*-Block. Dieser ist eine effizientere Implementierung des Cross-Stage-Partial (**CSP**) *Bottlenecks*, bei dem nur einen Teil der Kanäle transformiert und anschließend mit den unveränderten Kanälen konkateniert wird [7].

Neck Der *Neck* kombiniert die Merkmale aus unterschiedlichen Ebenen des *Backbones* und übergibt diese dem *Head*. Dazu werden die Merkmalskarten durch Methoden wie Upsampling und Konkatenation hochskaliert und zusammengeführt. Um den globalen Kontext effizient zu erfassen, wird am Ende des *Backbones* ein Spatial-Pyramid-Pooling-Fast (**SPPF**)-Block eingefügt. Dieser führt serielle Max-Pooling-Operationen aus und fasst die daraus entstehenden Merkmalskarten zusammen. Durch den folgenden Cross-Stage-Partial-with-Spatial-Attention (**C2PSA**) Block wird eine Aufmerksamkeitskarte berechnet, wodurch der Fokus auf relevante Bildregionen gelenkt wird [7].

Head Der *Head* verarbeitet die Merkmale in drei Ebenen mit weiteren *C3k2* und *Conv2D* Blöcken. Die Ebenen geben anschließend die **BB** Koordinaten zur Lokalisierung des Objekts, die Wahrscheinlichkeit für das Vorhandensein eines Objekts und die Klassenwahrscheinlichkeit an. Dabei verwenden die Ebenen unterschiedlich große Merkmalskarten und erstellen somit kleine, mittler und große **BB** für das Bild [7].

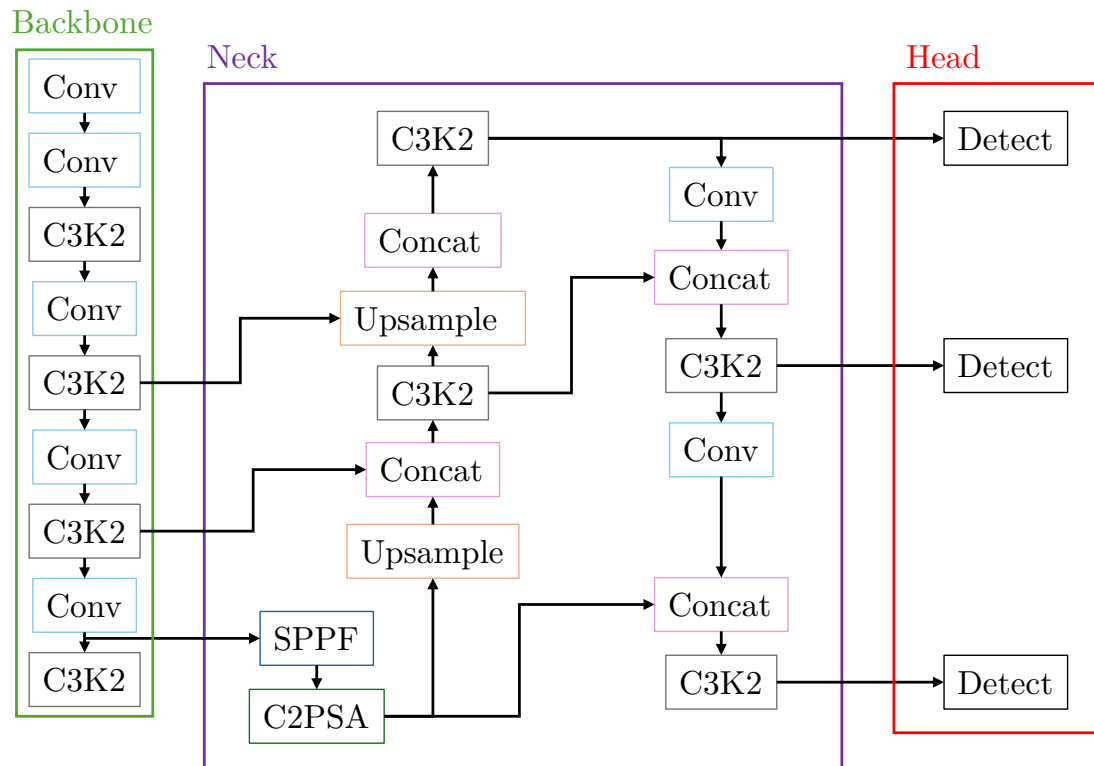


Bild 2.10. Netzwerkarchitektur Yolov11 nach [19]

Die Ausgabe des **Head** muss im Inferenzbetrieb (Testphase) in ein menschenlesbares Format decodiert werden, das heißt die vom Netz gelieferten Tensoren werden in Bildkoordinaten $(x1, y1, x2, y2)$, eine Klassen-ID und die zugehörige Wahrscheinlichkeit überführt.

PointPillar

Im Gegensatz zu früheren 3D-Detektoren schließt PointPillar (**PP**) die Lücke zwischen manuell designten Vogelperspektive / Birds-Eye View (**BEV**) Encodern und gelernten Encodern. Es lernt die Repräsentation *end-to-end* und verwendet dabei keine leistungsbeanspruchenden 3D-Faltungen, sondern lediglich Faltungen im 2D-Bereich. Dadurch ist es für diverse Daten flexibel einsetzbar und bietet zudem gute Performance. Das Netzwerk, dargestellt in ►Bild 2.11, liefert aus einer Punktwolke 3D-Bounding-Boxes

für Autos, Fußgänger und Fahrradfahrer. Es besteht dabei aus den drei Modulen *Pillar Feature Net*, *Backbone* und *Detection Head*.

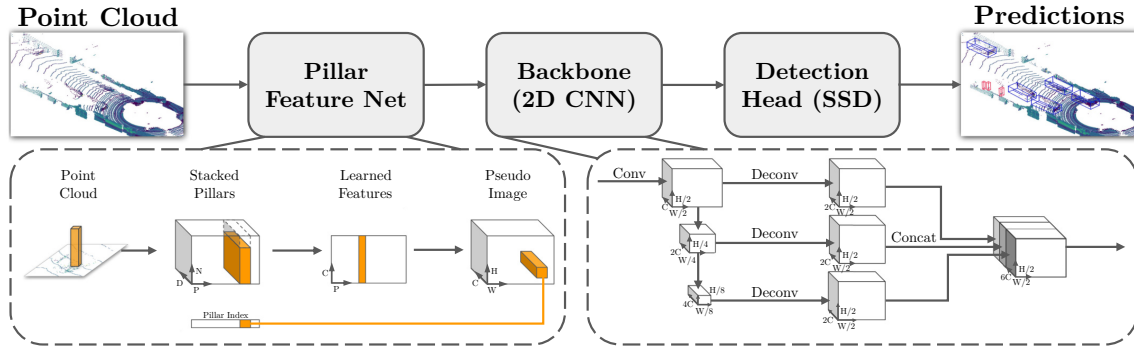


Bild 2.11. PointPillar Netzwerkarchitektur nach [8]

Pillar Feature Net Das *Pillar Feature Net* dient als trainierbares Encoder-Netzwerk, das die Punktwolke in ein dünn besetztes Pseudobild in der BEV überführt. Dazu wird die Punktwolke in der $x - y$ -Ebene gleichmäßig rasterisiert. Ein LiDAR-Punkt wird durch neun Merkmale D beschrieben: die Raumkoordinaten (x, y, z) und die Rückstreuintensität r , die Abstände zum arithmetischen Mittel der Punkte in derselben Säule (x_c, y_c, z_c) , sowie der Abstand relativ zum Säulenzentrum (x_p, y_p) . Aus diesen Größen entsteht ein kompakter Tensor $(\mathbb{R}^{D \times P \times N})$, wobei P die maximale Anzahl aktiver Säulen und N die maximale Punktzahl je Säule begrenzt. Die inhärente Sparsität der LiDAR-Daten wird genutzt, indem Säulen mit weniger als N Punkten mittels *Zero-Padding* aufgefüllt und Säulen mit mehr als N Punkten zufällig heruntergesampt werden. Eine lineare Projektion (äquivalent zu 1×1 -Faltungen), gefolgt von Batch-Normalisierung und ReLU, transformiert die Daten zu einem Tensor $(\mathbb{R}^{C \times P \times N})$. Eine anschließende Max-Aggregation über die Punktdimension liefert Säulenmerkmale $(\mathbb{R}^{C \times P})$, die abschließend über die bekannten Säulenindizes in das BEV-Gitter zurückgestreut werden und so ein Merkmalsbild $\mathbb{R}^{C \times H \times W}$ ergeben, wobei H und W die Höhe beziehungsweise Breite des Canvas angeben [8].

Backbone Das *Backbone* besteht aus zwei Teilnetzen. Der *Top-Down-Zweig* extrahiert Merkmale auf progressiv kleiner werdenden räumlichen Auflösungen und bildet in Bild 2.11 die linke Seite der *Backbone*-Architektur. Jeder Block besteht aus mehreren 3×3 Conv-BN-ReLU-Schichten, wobei die erste Schicht mit der Schrittweite zwei arbeitet, sich die Auflösung (H, W) also jeweils halbiert und die Kanalanzahl C verdoppelt. Der zweite, *Up-Sampling-Zweig* hebt mit transponierten Faltungen die Merkmale auf eine gemeinsame Auflösung, bevor sie entlang der Kanaldimensionen konkateniert werden [8].

Detection Head Als *Detection Head* wird ein *Single Shot Detektor* zur 3D-Erkennung verwendet. Dabei arbeitet der Kopf ankergestützt in der BEV und platziert pro Gitter-

zelle zwei orientierte Anker ($0^\circ/90^\circ$) mit klassenspezifischen Standardabmessungen und z-Referenzlage. Als Anker bezeichnet man vordefinierte, orientierte 3D-Referenzboxen, relativ zu denen **PP** pro Zelle Klassenscores und Abweichungen für Lage, Ausdehnung und Orientierung vorhersagt. In der Inferenz wird mittels Score-Filter und Non-Maximum-Suppression (**NMS**) passende **BB** ausgewählt.

DeepStream

Das NVIDIA-DeepStream-**SDK** stellt vier unterschiedliche Tracker-Backends bereit und ist in das Open-Source-Multimediaframework *GStreamer* integriert. In *GStreamer* werden Verarbeitungsschritte als *Plugins* modelliert und zu einem Graphen verbunden, der eine Pipeline aus Quellen, Decodern, Filtern, Encodern und Senken abbildet. Darüber hinaus übernimmt *GStreamer* die Pufferverwaltung, Ablaufplanung und Synchronisation der Datenströme. Für die Objektdetektion (Inference) und das anschließende Multi-Objekt-Tracking stellt NVIDIA eigens entwickelte, GPU-beschleunigte Plugins zur Verfügung, siehe ►Bild 2.12 [15].

... ! nvv4l2decoder ! nvinfer ! nvtracker ! nvosd ! ...

Bild 2.12. Ausschnitt Pipeline Video Dekodierung -> Inferenz -> Tracking
-> Visualisierung

Nachfolgend werden die vier Tracker **IoU**, **NvSORT**, **NvDeepSORT** und **NvDCF** vorgestellt.

IoU Zur Assoziation erkannter Objekte in aufeinanderfolgenden Frames nutzt der *IoU-Tracker* einen Greedy-Algorithmus. Die Zuordnung basiert auf dem Intersection-over-Union (**IoU**) Wert der 2D **BB** und wird durch Robustheitsmechanismen, wie *Late-Activation* und *Shadow-Tracking*, abgesichert. Mit der *Late-Activation* wird versucht, falsch positive Objekterkennung zu unterdrücken, indem ein neu erkanntes Objekt über mehrere Frames hinweg erkannt werden muss, um als aktiver Track zu zählen. *Shadow-Tracking* ermöglicht es dagegen, ein Objekt für einen definierten Zeitraum weiterzuverfolgen, bis der Detektor es wieder erkannt hat oder ein Schwellwert zur Deaktivierung des Tracks erreicht wird [15].

NvSORT Im Gegensatz zum **IoU-Tracker** setzt *NvSORT* einen **KF** als Zustandsschätzer für die Objekte ein. Darüber hinaus erfolgt die Datenzuordnung in mehreren Stufen auf Basis räumlicher Nähe und Detektionssicherheit [15].

NvDeepSORT Der *NvDeepSORT-Tracker* baut auf **NvSORT** auf, verwendet allerdings **DL**. Ein re-identification (**Re-ID**) Neuronales Netzwerk extrahiert Merkmale für jedes Objekt und vergleicht diese mittels der Kosinus-Distanzmetrik. Anschließend wird zur Datenzuordnung wieder die räumliche Nähe und zusätzlich die **Re-ID** Ähnlichkeit verwendet [15].

NvDCF Der *NvDCF-Tracker* verwendet einen discriminative-correlation-filter (**DCF**), um einen Korrelationsfilter für das Zielobjekt zu lernen und anschließend das gleiche Objekt im nächsten Frame mit dem gelernten Filter wiederzufinden. Im Gegensatz zu **NvSORT** und **NvDeepSORT** erhält der **KF** neben den erkannten **BB** auch die Lokalisierungsergebnisse aus dem visuellen Trackingmodul. Dieses ist in ►Bild 2.13 dargestellt und besteht aus den zwei Schritten Lokalisierung und Aktualisierung. Im Lokalisierungsschritt wird jede detektierte **BB** aus dem Bild ausgeschnitten und als Suchbereich verwendet, wobei der Suchbereich etwas größer gewählt wird als die ursprüngliche **BB**, um auch mögliche Bewegungen des Objekts zwischen den Frames zu berücksichtigen. In den Suchbereichen werden Merkmale in verschiedenen Kanälen mittels Histogramm-of-Oriented-Gradients (**HOG**) und/oder *ColorNames*. Dabei eignet sich **HOG** zur Kantendetektion und *ColorNames* zur Unterscheidung von Farben. Beide Varianten werden von Tracker unterschiedlich gewichtet, um das Objekt vom Hintergrund zu trennen. Die Korrelationsfilterkanäle werden anschließend aufsummiert, wodurch die Korrelationsantwortkarte in ►Bild 2.14 entsteht. In dieser steht das Maximum für das neue Objektzentrum. Dieser wird an den Datenassoziation übergeben, wodurch Tracking auch ohne Detektion möglich ist. Im Aktualisierungsschritt wird aus dem Suchbereich eine räumliche Zuverlässigkeitsmaske geschätzt, die Objektpixel betont und den Hintergrund unterdrückt. Auf Basis der maskierten Merkmale werden Filter und Kanalgewichte per **DCF**-Optimierung neu berechnet und anschließend mithilfe eines exponentiell gleitenden Durchschnitts mit dem bisherigen Modell verschmolzen. [14] [15]

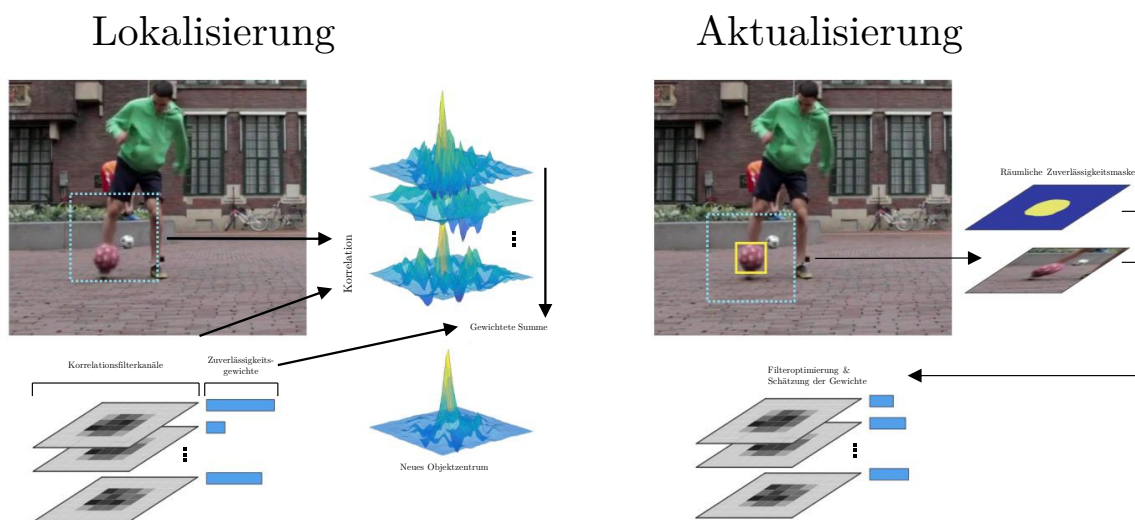


Bild 2.13. DCF Ablauf nach [14]

Das gelbe Kreuz in Bild 2.14 markiert das Maximum der Korrelationsantwort und damit die geschätzte Objektposition. Die lila Kreuze kennzeichnen die Zentren der BB. Die Korrelationswerte an diesen Positionen messen die visuelle Ähnlichkeit zum gelernten Filter - je höher der Wert, desto plausibler die Objektpräsenz an dieser Stelle [14] [15].



Bild 2.14. Beispiel DCF-Tracking [15]

MCTrack

Dieser 3D-Tracker zeichnet sich, wie bereits in der Einleitung erwähnt, durch seine SOTA Performance in diversen Multi-Object-Tracking (MOT) Benchmarks aus. Ermöglicht wird dies durch das Umwandeln der Datensätze in ein Standardformat, welches in den Eingang des Algorithmus gegeben wird. Die Funktionsweise ist in ►Bild 2.15 dargestellt und wird im folgenden Absatz beschrieben [25].

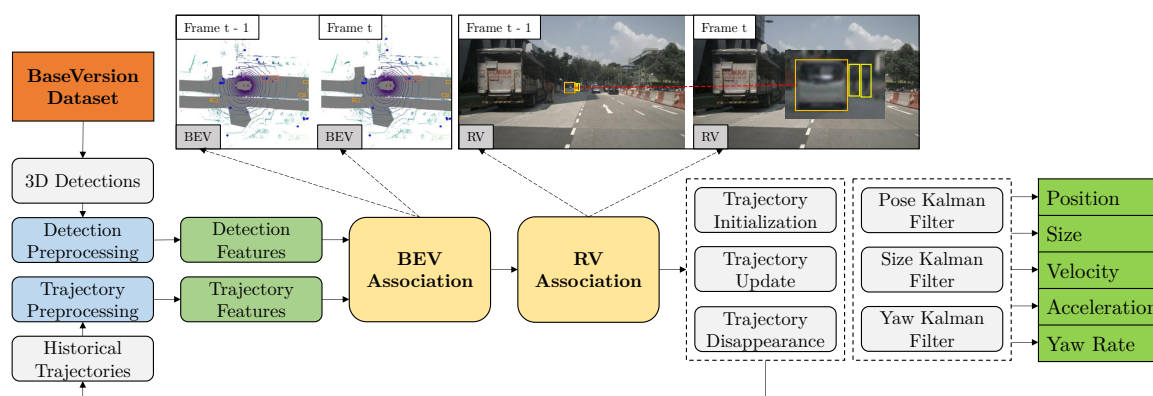


Bild 2.15. MCTrack Pipeline nach [25]

Der Tracker benötigt die 3D BB Dimensionen und Pose im Weltkoordinatensystem. Dabei wird eine Rotation nur um die z-Achse vorausgesetzt. Des Weiteren wird die Kategorie und Erkennungswahrscheinlichkeit der BB benötigt. Anschließend werden die BB in

die BEV Ebene projiziert. In dieser Ebene werden die projizierten Features mit den bestehenden Trajektorien assoziiert. Im nächsten Schritt werden die restlichen Erkennungs- und Trajektorien-BB in die Range"View (RV)-Ebene projiziert. Hier findet eine zweite Zuordnung statt, bevor die Zustände der Trajektorien und des KF aktualisiert werden. Für den nächsten Frame werden die aktuellen Trajektorien rückgeführt und der Tracker liefert als Ergebnis folgende Dimensionen: [25]

- Position
- Dimension
- Geschwindigkeit
- Beschleunigung
- Drehrate

Wie in Bild 2.15 zu erkennen, werden drei getrennte KF für das Bewegungsmodell verwendet. Für die Position (x, y) wird ein konstantes Beschleunigungsmodell initialisiert, wohingegen für die Breite und Länge (l, w) , sowie der Orientierung Θ ein konstantes Geschwindigkeitsmodell benutzt wird. Bevor wir die Kostenfunktion zur Zuordnung von BB und Trajektorien nutzen, werden die Trajektorien vorwärts und die Detektionen rückwärts propagiert. Die Vorwärtsrechnung ist in ►Gleichung 2.11 und die Rückwärtsrechnung in ►Gleichung 2.12 dargestellt [25].

$$T_{\tau}^j = \mathcal{F}(x_{\tau-1}^t, y_{\tau-1}^t, v_{x,\tau-1}^t, v_{y,\tau-1}^t, \Delta\tau) \quad (2.11)$$

$$D_{\tau-1}^i = \mathcal{F}^{-1}(x_{\tau}^d, y_{\tau}^d, v_{x,\tau}^d, v_{y,\tau}^d, -\Delta\tau) \quad (2.12)$$

Dabei steht $\Delta\tau$ für den zeitlichen Abstand zwischen dem aktuellen und vorherigem Frame. Die Trajektorien T_{τ} werden unter Verwendung des Bewegungsmodells $\mathcal{F}(\bullet)$ berechnet, während die Detektionen $D_{\tau-1}$ durch das inverse Bewegungsmodell ermittelt werden. Aus den beiden Vorhersagen wird als nächstes die Übereinstimmung $\mathcal{C}$ berechnet um die Kostenfunktion $\mathcal{L}_{cost}$ in ►Gleichung 2.13 zu erhalten [25].

$$\mathcal{L}_{cost} = \beta * \mathcal{C}(D_{\tau}, T_{\tau}) + (1 - \beta) * \mathcal{C}(D_{\tau-1}, T_{\tau-1}) \quad (2.13)$$

Bei β handelt es sich um einen Gewichtungsfaktor $\beta \in [0,1]$, der die Kosten der vorwärts prognostizierten Zuordnung $\mathcal{C}(D_{\tau}, T_{\tau})$ gegenüber der rückwärts prognostizierten Zuordnung $\mathcal{C}(D_{\tau-1}, T_{\tau-1})$ gewichtet. Die Zuordnungsfunktion $\mathcal{C}$ ist eine Abkürzung für den Rotation-aware Generalized Distance Intersection-over-Union (Ro_GDIoU) Algorithmus. Die Funktionsweise ist in ►Bild 2.16 schematisch dargestellt und wird nachfolgend beschrieben.

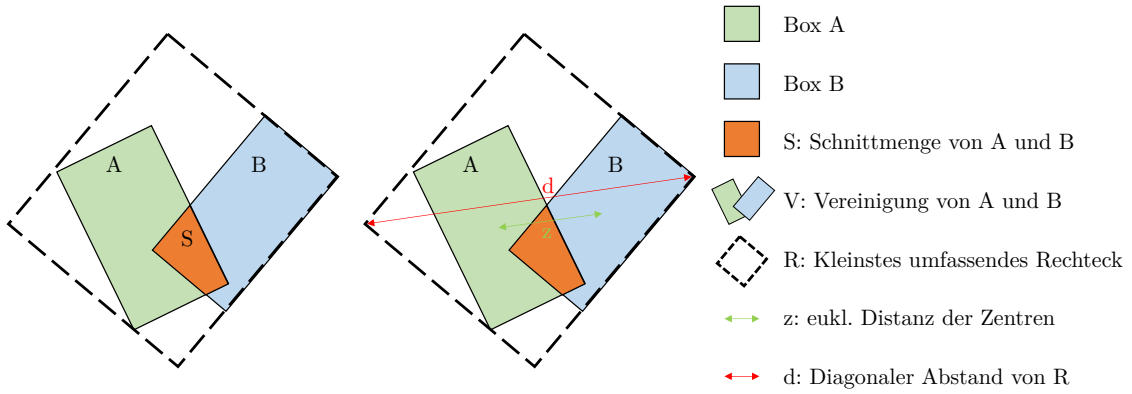


Bild 2.16. Darstellung der `Ro_GDIOU` Berechnung nach [25]

Zunächst werden die Schnittmenge S , die Vereinigung V und das kleinste, umfassende Rechteck R der beiden `BB` (A, B) berechnet. Anschließend wird die diagonale Distanz von R , d , und die euklidische Distanz der beiden `BB` Zentren z berechnet. In ►Gleichung 2.14 ist die Berechnung des `Ro_GDIOU` dargestellt. Dabei werden die Ergebnisse der `IoU`- und euklidischen Distanzberechnung durch die beiden Faktoren (ω_1, ω_2) gewichtet:

$$Ro_GDIOU = \frac{S}{V} - \omega_1 * \frac{R - V}{R} - \omega_2 * \frac{z^2}{d^2} \quad (2.14)$$

2.2 KITTI-Datensatz

Der *KITTI-Datensatz* wurde in Karlsruhe und Umgebung mit dem *AnniWay*-Autonomen-Fahrzeug, abgebildet in ►Bild 2.17 (linke Seite), aufgezeichnet. Er stellt neben den 360° LiDAR Punktwolken und 2D `RGB`-Bildern auch die Transformationsmatrixen und Posen in Weltkoordinaten dar. Neben dem Datensatz sind mehrere Benchmarks erstellt worden, in welchen Ground-Truth (`GT`) Detektionen und Tracks veröffentlicht sind. Diese werden zur Evaluierung verschiedener Methoden verwendet. Auf der rechten Seite von Bild 2.17 sind die Transformationen und das 3D `BB` Format abgebildet.

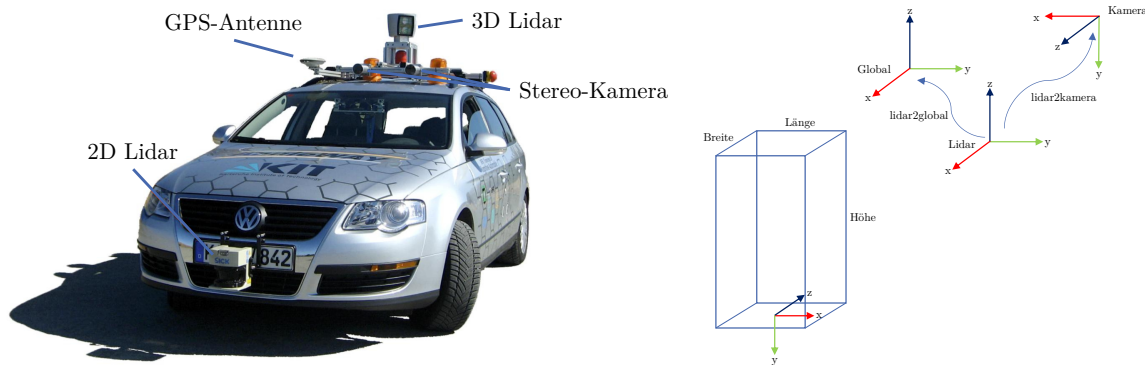


Bild 2.17. Aufzeichnungsplattform, Koordinatentransformation und 3D-Bounding-Box-Format nach [6]

Der Erkennungsdatensatz besteht aus 7481 Trainingsbildern und 7581 Testbildern. Die Klassenverteilung ist in ►Bild 2.18 dargestellt. Diese repräsentiert zwar die echte Verteilung im Straßenverkehr, erschwert aber das Trainieren der weniger vertretenen Klassen.

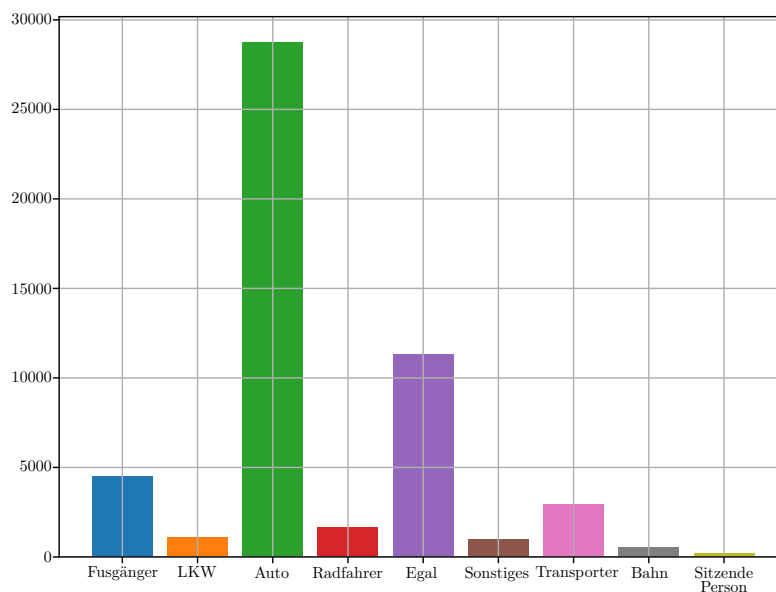


Bild 2.18. Klassenverteilung GT der Trainingsbilder

Objekte die mit *egal* gekennzeichnet sind, werden für die Evaluation im Benchmark ignoriert, da die Objekte beispielsweise für den Laserscanner zu weit entfernt sind. Zu den Objekten mit dem Label *sonstiges* gehören zum Beispiel Anhänger oder Segways.

Neben den BB werden auch Instanzsegmentierungsmasken, siehe ►Bild 2.19, zur Verfügung gestellt [4]. Diese werden für die Klassen *Fußgänger* und *Auto* manuell annotiert und beschreiben nicht nur BB-Objekte.



Bild 2.19. KITTl Instanz+ BB-Annotationen

2.3 Tracking-Evaluierung

Zum Vergleich verschiedener Tracking-Algorithmen werden Evaluationsmetriken eingesetzt, die die Ergebnisse dem **GT** gegenüberstellen. Ziel ist die Bewertung von **Lokalisierung**, **Detektion** und **Assoziation** über einen zeitlichen Verlauf. Wie in ►Bild 2.20 dargestellt, setzten verschiedene Metriken den Schwerpunkt nur auf eine Teilaufgabe des Trackings. Gezeigt werden die Ergebnisse von drei Trackern mit steigender Detektionsgenauigkeit und gleichzeitig sinkender Assoziationsgenauigkeit im Vergleich zur **GT**. Tracker A liefert nur einen einzigen vorhergesagten Track, was zu einer hohen Assoziationsgenauigkeit führt. Tracker B erkennt zwar mehr Objekte, teilt diese aber auf zwei Tracks auf, wodurch die Assoziationsgenauigkeit sinkt. Tracker C deckt die gesamte Trajektorie ab und erreicht damit die höchste Detektionsgenauigkeit, teilt diese jedoch in vier verschiedene Tracks auf, was die Assoziationsgenauigkeit am stärksten verringert. Aus diesem Grund wurde die Higher-Order-Tracking-Accuracy (**HOTA**)-Metrik entwickelt, die beide Teilaufgaben gleichermaßen berücksichtigt. In den folgenden Absätzen werden die wichtigsten Metriken vorgestellt [13].

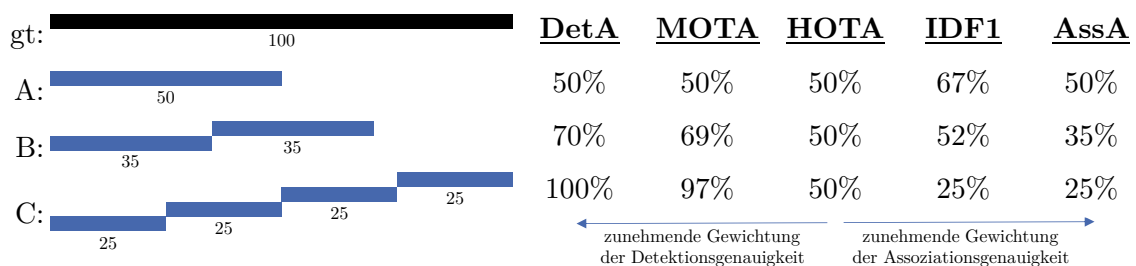


Bild 2.20. Tracking Beispiel zur Verdeutlichung des Unterschieds zwischen den Evaluationsmetriken nach [13]

Zuerst werden allerdings noch wichtige Begrifflichkeiten erläutert:

- **True-Positive (TP)**: Ein vorhergesagtes Objekt stimmt mit einem **GT** Objekt überein [13].

- **False-Positive (FP)**: Ein vorhergesagtes Objekt stimmt mit keinem **GT** Objekt überein [13].
- **False-Negative (FN)**: Ein **GT** Objekt wird nicht erkannt oder assoziiert [13].
- **Identity-Switch (IDSW)**: Ein Tracker tauscht fälschlicherweise Objekt-IDs oder reinitialisiert ein Objekt mit einer anderen Objekt-ID als ursprünglich vergeben [13].

2.3.1 Lokalisierung - **LocA**

Die Localization-Accuracy (**LocA**) misst die räumliche Übereinstimmung zwischen den vorhergesagten und den **GT BB**. Dazu wird wie in ►Bild 2.21 dargestellt, der Lokalisierungs-**IoU**-Wert aus der Schnittfläche und der kombinierten Fläche berechnet. Um den globalen *Lok-IoU* Wert zu berechnen, wird er über alle **TP** gewichtet, wie in ►Gleichung 2.15 zu sehen: [12]

$$LocA = \frac{1}{|TP|} \sum_{c \in TP} Loc - IoU(c) \quad (2.15)$$

2.3.2 Detektion - **DetA**

Im Gegensatz zum **LocA** bewertet die Detection-Accuracy (**DetA**) die Fähigkeit eines Trackers, Objekte zu erkennen. Es wird also nicht die räumliche Übereinstimmung der **BB** bewertet, sondern nur ob ein Objekt erkannt wird oder nicht. Dazu wird ein Schwellwert α (z.Bsp.: $Loc - IoU > 0,5$) eingeführt, ab dem die **BB** als erkannt gilt. Für den Fall das eine **BB** mehrere **GT** Objekte schneidet, wird der *Hungarian*-Algorithmus verwendet, um eine Eins-zu-Eins Zuordnung zu gewährleisten. Detektionen über dem Schwellwert werden als **TP** gezählt, während die restlichen als **FP** klassifiziert werden. Die übrigen **GT** Objekte, die nicht erkannt wurden, werden als **FN** gezählt. Die **DetA** wird in ►Gleichung 2.16 berechnet: [12]

$$DetA = \frac{|TP|}{|TP| + |FN| + |FP|} \quad (2.16)$$

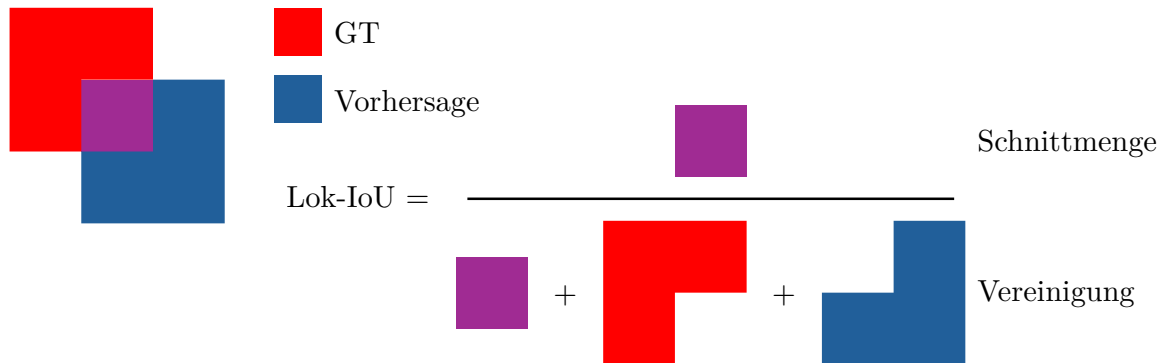


Bild 2.21. Lokalisierungs-IoU nach [12]

2.3.3 Assoziation - AssA

Wie gut ein Tracker dasselbe Objekt über mehrere Frames verbindet, gibt der Association-Accuracy ([AssA](#)) Wert an. Dazu wird die Überlappung einer vorhergesagte Detektion und eine [GT](#) Detektion, beide wie bei [DetA](#) durch den *Hungarian*-Algorithmus ausgewählt, über den gesamten Zeitraum verglichen. Dieser Prozess ist in ►Bild 2.22 dargestellt. Die Überlappung von zwei Tracks wird als True-Positive-Associations ([TPA](#)) bezeichnet. Die Tracks, die nicht zu einem [GT](#) Track zugeordnet wurden, sind False-Positive-Associations ([FPA](#)) und die übrigen [GT](#) Tracks sind False-Negative-Associations ([FNA](#)). In Bild 2.22 ist das ausgewählte Paar (Detektion & [GT](#)) rot eingerahmt. Die [TPA](#) sind grün, die [FPA](#) gelb und die [FNA](#) braun eingerahmt. Die [AssA](#) wird, über alle [TP](#) Paare gemittelt, mit der ►Gleichung 2.17 berechnet:[12]

$$AssA = \frac{1}{|TP|} \sum_{c \in TP} \frac{|TPA(c)|}{|TPA(c)| + |FNA(c)| + |FPA(c)|} \quad (2.17)$$

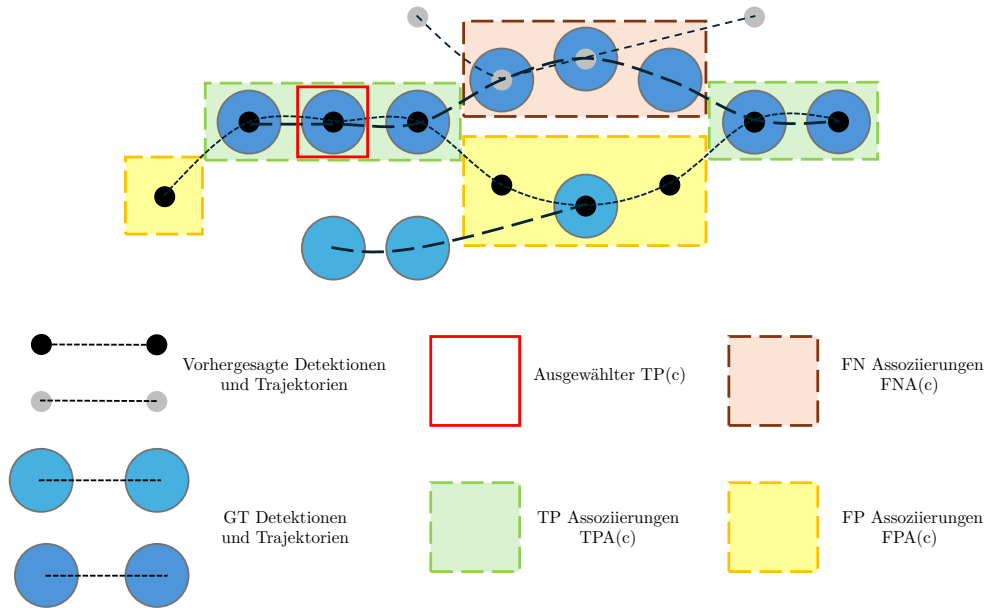


Bild 2.22. AssA Berechnungsbeispiel nach [12]

2.3.4 MOTA

Die Multiple-Object-Tracking-Accuracy (MOTA) Metrik misst IDSW, indem die aktuell vorhergesagte ID mit der vorherigen ID verglichen wird. Ändert sich diese im Gegensatz mit der vorherigen TP ID desselben GT, wird der MOTA Wert in Gleichung 2.18 schlechter. Allerdings prüft die MOTA Metrik nicht, wenn dieselbe ID einer anderen GT ID zugeordnet wird [13].

$$MOTA = 1 - \frac{|FN| + |FP| + |IDSW|}{|gtDet|} \quad (2.18)$$

2.3.5 IDF1

Im Kontrast zu MOTA, prüft die Identification-F1 (IDF1) Metrik nicht auf der Detektionsebene, sondern auf der Trajektorienebene. Wenn die Überlappung zwischen predicted-Trajectories (prTrajs) und ground-truth-Trajectories (gtTrajs) größer als ein Schwellwert ist, handelt es sich um Identity-True-Positives (IDTPs). Identity-False-Negatives (IDFNs) und Identity-False-Positives (IDFPs) sind die übrig gebliebenen ground-truth-Detections (gtDets) und predicted-Detections (prDets), welche nicht überlappen, aber einem Track entweder zugeordnet oder nicht zugeordnet werden können [13].

$$IDF1 = \frac{|IDTP|}{|IDTP| + 0,5|IDFN| + 0,5|IDFP|} \quad (2.19)$$

2.3.6 HOTA

Für die Berechnung des **HOTA** Werts fließen alle drei Metriken **LocA**, **DetA** und **AssA** ein. Dazu wird zunächst $HOTA_\alpha$ mit ►Gleichung 2.20 aus dem Schwellwert α - vergleiche Gleichung 2.15 - berechnet [13].

$$HOTA_\alpha = \sqrt{DetA_\alpha * AssA_\alpha} \quad (2.20)$$

$HOTA_\alpha$ wird mit dem Schwellwert α zwischen 0,05 und 1 integriert, woraus sich **HOTA** in ►Gleichung 2.21 ergibt [13].

$$HOTA = \int_{0 < \alpha \leq 1} HOTA_\alpha = \frac{1}{19} \sum_{0,05}^{0,95} HOTA_\alpha | \Delta\alpha = 0,05 \quad (2.21)$$

►Bild 2.23 zeigt die vorgestellten Metriken zur Berechnung des **HOTA** Werts anhand einer Trackingsequenz. Auf der horizontalen Achse wird der **Loc-IoU**-Schwellwert α zwischen 0,00 bis 1,00 variiert und der Flächeninhalt der Graphen ergibt das jeweilige Endergebniss für die Sequenz. In der **Evaluierung** von Trackingsequenzen werden noch weitere Graphen und Details erläutert.

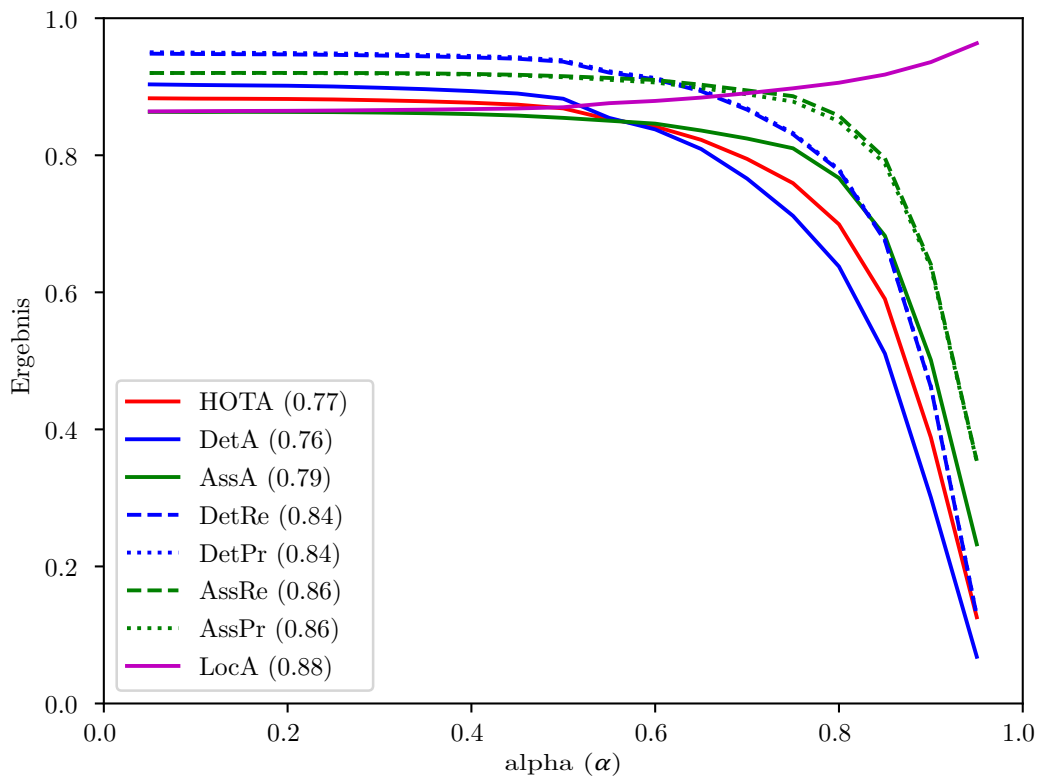


Bild 2.23. Evaluierungsmetriken einer Trackingsequenz